Failing with Purpose: Dangling Coverage-Guided Negative Test Generation from a Mechanized P4 Type System

JAEHYUN LEE, KAIST, South Korea
SEOKHUN JEONG, KAIST, South Korea
SUKYOUNG RYU, KAIST, South Korea

A type checker must reject ill-typed programs in addition to accepting well-typed programs. *Negative type checker tests*, programs expected to be rejected, validate that a type checker enforces the language's typing rules as intended. We focus on negative type checker tests for P4, a domain-specific language for programmable network devices, whose type system encodes design principles and hardware constraints of the network dataplane. Failing to reject an ill-typed P4 program risks violating these principles and constraints, leading to unexpected errors. A comprehensive negative test suite covering subtle and diverse ill-typed conditions is thus important. However, constructing comprehensive negative tests is challenging: the negative input space lacks systematic characterization, and existing P4 program generators do not target subtle type errors.

This paper addresses the problem in three steps. (i) We mechanize the P4 type system using the SpecTec framework. Unlike the informal official P4 specification, the mechanized type system is formal and machine-readable. Mechanization enables a systematic analysis of the type system. (ii) Across the mechanized type system, we identify *dangling premises*, which are premises in the typing rules that, when violated, cause type errors. Based on them, we propose *dangling coverage*, a novel metric for quantifying negative test coverage. (iii) Finally, we implement a coverage-guided fuzzer that mutates well-typed P4 programs into ill-typed programs that increase dangling coverage. Our method identifies 939 dangling premises that characterize distinct ill-typed conditions in the P4 type system. The fuzzer generates a negative test suite achieving 33.02%p higher dangling coverage than the existing P4C reference compiler's test suite. The generated tests also reveal 29 previously unknown bugs in the compiler frontend, demonstrating the effectiveness of both the coverage metric and the fuzzer. The tests generated by our fuzzer are now integrated into the P4C test suite.

CCS Concepts: • **Software and its engineering** → **Specification languages**; **Software testing and debugging**.

Additional Key Words and Phrases: P4, type system, mechanization, negative testing, fuzzing

ACM Reference Format:
Jaehyun Lee, Seokhun Jeong, and Sukyoung Ryu. 2026. Failing with Purpose: Dangling Coverage-Guided Negative Test Generation from a Mechanized P4 Type System. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE081 (July 2026), 21 pages. https://doi.org/10.1145/3797109

1 Introduction

Type checkers are two-sided. In addition to accepting well-typed programs, they must also reject ill-typed programs. When a type checker erroneously rejects a well-typed program, we call it a *precision bug*. Conversely, when it fails to reject an ill-typed program, it is a *soundness bug*. Positive and negative tests are used to validate that a type checker does not exhibit precision and soundness bugs, respectively. In other words, negative tests are intentionally ill-typed programs designed to validate that the type checker correctly rejects them.

Authors' Contact Information: Jaehyun Lee, 99jaehyunlee@kaist.ac.kr, KAIST, South Korea; Seokhun Jeong, sraccoon@kaist.ac.kr, KAIST, South Korea; Sukyoung Ryu, sryu.cs@kaist.ac.kr, KAIST, South Korea.

ACM 2994-970X/2026/7-ARTFSE081
https://doi.org/10.1145/3797109

Well-designed negative tests are important for a type checker. This holds true for P4 [6], a statically typed domain-specific language (DSL) for programming packet-processing network devices such as switches and NICs. P4 enables network developers to define custom network protocols, including packet header structures, parsers that extract headers from an input bit-vector, and packet forwarding logic in an imperative style. The P4 type system reflects the core design principles and hardware constraints of the network programming domain. For instance, it imposes various domain-specific restrictions on bit-level arithmetic to match the hardware capabilities of network devices. Additionally, the P4 type system incorporates advanced features, such as implicit cast insertion, compile-time evaluation, and type inference. These restrictions and advanced features give rise to subtle ill-typed conditions that a P4 type checker must check. An underdeveloped negative test set may fail to uncover these subtleties. As a result, a P4 type checker may overlook type system intricacies that manifest as unexpected runtime errors in packet processing.

To assess whether a P4 type checker test suite is comprehensive, we need to establish a connection between the language specification and the test suite. The core P4 language ecosystem is shaped by two bodies: the P4 specification [18] and the P4C reference compiler [19]. The official P4 specification defines the syntax, type system, and dynamic semantics in informal prose. Alongside this is P4C, an open-source reference compiler, with a test suite including both positive and negative type checker tests. The test suite helps validate that P4 language implementations adhere to the specification. In particular, a comprehensive negative test suite should explicitly trigger all the ill-typed conditions described in the specification.

However, there are missing links between the specification and the test suite, where what is described as ill-typed in the specification is not explicitly tested. For example, regarding bit shift operations, Section 8.9.2 of the P4 specification states:

> [...] the right operand of shifts must be either an expression of type **bit**<s> or a compile-time known value that is a non-negative integer. [...] Shifting with a negative amount does not have a clear semantics: the P4 type system makes it illegal to shift with a negative amount.

As per the specification, the below program, **shift-ill.p4**, should be rejected.

```
header Hdr { int<8> src; }
bit<32> f(in bit<32> x, in Hdr hdr) { return x << hdr.src; }
```

It defines a header type `Hdr` with a field `src` of fixed-width signed integer type **int**<8>. In function `f`, the field `hdr.src` is used as the right operand of shift. This violates the specification, since `hdr.src` is neither an unsigned integer type (**bit**<S>) nor a compile-time constant. Surprisingly, the P4C test suite lacks a negative test targeting this restriction. Relatedly, a recently confirmed soundness bug [34] revealed that P4C failed to reject an even more erroneous violation, shifting by a string literal, suggesting broad gaps between the negative test suite and the specification.

To identify such bugs and increase confidence in language implementations, fuzzing can be employed to create test programs. Yet, little effort has been made toward generating negative type checker tests for P4. Gauntlet [27] and P4Fuzz [3] apply random program generation to find bugs in P4C. However, their generation strategies focus on producing well-typed programs that can get past the compiler frontend and reach later compiler stages. As a result, they prioritize testing compiler optimization and dynamic semantics over static type checking. To our knowledge, no existing tool systematically targets the generation of negative tests for the P4 type checker.

Generating negative type checker tests for P4 presents several challenges. First, as mentioned earlier, there exist inherent gaps between the P4 specification and the P4C test suite. The specification informally describes ill-typed conditions, but they are not systematically enumerated. In

```
rule Expr_ok/bine-shift:
  C |- BinE binop expr_l expr_r : BinE binop exprIR_l exprIR_r `( typeIR_l )
  ----
  -- if binop <- [ SHL, SHR ] ;; 1
  -- Expr_ok: C |- expr_l : exprIR_l ;; 2
  -- Expr_ok: C |- expr_r : exprIR_r ;; 3
  ----
  -- if typeIR_l = $typeof(exprIR_l) ;; 4
  -- if typeIR_r = $typeof(exprIR_r) ;; 5
  -- if $compatible_shift(typeIR_l, typeIR_r) ;; 6
  -- if (($is_intt(typeIR_r) \/ $is_fintt(typeIR_r)) => $is_ctk_non_neg(C, exprIR_r)) ;; 7
```

Fig. 1. Simplified shift expression typing rule mechanized in the SpecTec meta-language.

parallel, tests in the P4C test suite are not classified by the kind of type error they target. The tests are largely driven by intermittent bug reports, where tests are added to ensure that the reported bugs are fixed. One may proxy the ill-typed conditions by inspecting the P4C compiler source code. However, P4C adopts a nano-pass architecture [7], in which type checking logic is dispersed across various frontend passes. This makes it difficult to trace the compiler source code back to the specification level. A systematic approach is needed to enumerate all ill-typed conditions in the specification and to link them to the tests in the test suite.

Second, generating ill-typed programs, as simple as it may sound, gets difficult when the goal is to trigger diverse, subtle type errors. Traditional random generation strategies based on context-free grammars tend to produce trivially incorrect code, such as ones with free identifier errors. However, a comprehensive test suite must capture subtleties around advanced language features and domain-specific constraints that developers may overlook. An effective test generation strategy is necessary to produce interesting ill-typed programs covering a wide range of ill-typed conditions.

To systematically enumerate the ill-typed conditions, we utilize a language mechanization framework to mechanize the formal P4 type system specification. Language mechanization frameworks provide a meta-language to define the syntax and semantics of a programming language. Unlike conventional prose specification, the mechanized specification written in a meta-language is machine-manipulable. Thus, the mechanized specification can be automatically transformed, analyzed, and even executed. Among various frameworks, including the K framework [26], PLT-Redex [13], Ott [28], and ESMeta [2], we adopt the recently developed SpecTec [32] framework.

Using the SpecTec meta-language, we define the P4 syntax and type system in terms of inference rules. Fig. 1 shows a simplified version of the mechanized typing rule for shift operations. It type checks an expression (`BinE binop expr_l expr_r`) under the typing context `C`, concluding that the expression is well-typed when the seven premises hold. Note that the last premise formally expresses the requirement in the specification: when the right operand's type (`typeIR_r`) is a signed integer type (`$is_intt`, `$is_fintt`), it must be compile-time known and non-negative (`$is_ctk_non_neg`). We extend the SpecTec framework to be able to execute the typing rules. Thus, given a P4 program, we can run the mechanized typing rules to check whether the program is well-typed.

Based on the mechanized specification, we define a coverage metric over the negative input space. Premises of the inference rules collectively define the conditions under which a P4 program is well-typed. However, inference rules do not define the error conditions. Instead, what is left undefined by the premises, i.e., their violations, implicitly defines ill-typed conditions. For instance, violation of the seventh premise in Fig. 1 represents an ill-typed condition. With this observation, we collect *dangling premises* – typing rule premises whose violations correspond to ill-typed conditions. They characterize the ill-typed conditions specified in the type system. We define a coverage metric over

the dangling premises, called *dangling coverage*. Since the mechanized specification is executable, we can run the type system against a P4 program and measure its dangling coverage. This bridges the gap between the specification and the negative type checker test suite.

Finally, to generate ill-typed programs triggering subtle type errors, we employ coverage-guided fuzzing that mutates well-typed programs into ill-typed ones. P4C holds a relatively rich set of over 1,000 positive tests exhibiting diverse advanced P4 language features. Type errors are typically one-off deviations from these well-typed programs. So, we leverage the positive test suite as a seed for our fuzzer, which mutates well-typed programs to explore the negative input space.

Our contributions in this work are follows:

- We define a mechanized specification of the P4 type system using the SpecTec framework.
- We introduce dangling coverage, a systematic metric for characterizing ill-typed conditions in the P4 type system.
- We apply dangling coverage-guided fuzzing to generate negative tests that explore previously untested areas of the P4 type system. This yields a negative test suite with 59.32% dangling coverage, exceeding the baseline of 26.30% from the P4C test suite.
- We discover 29 bugs in the P4C compiler frontend.
- The generated negative tests are now integrated into the P4C test suite.

2 Overview

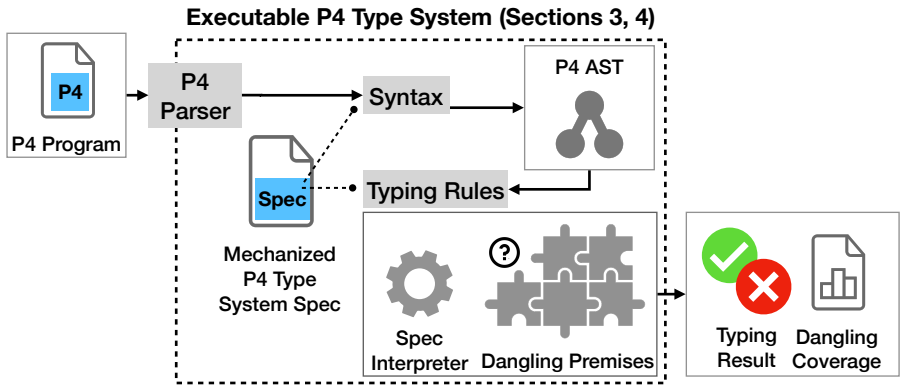
2.1 Mechanized P4 Type System

We first mechanize the P4 syntax and type system in the SpecTec meta-language (Fig. 2a). With mechanization, the type system can be treated as data, on which we apply automatic transformation, analysis, and execution in the later sections. Details of the mechanization are discussed in Section 3.

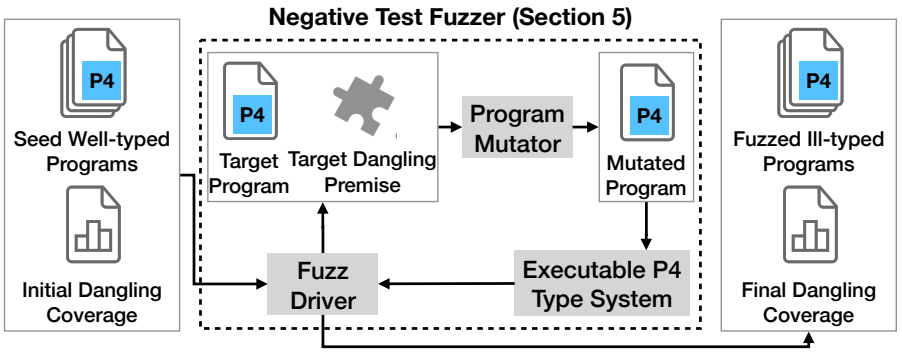
We extend SpecTec to identify dangling premises, which are typing rule premises that cause type errors when violated. Recall the shift expression typing rule in Fig. 1. The first premise, `if binop <- [SHL,SHR]`, restricts the rule to shift operators. It has counterparts in other typing rules for other operators like `if binop <- [PLUS,MINUS]`. Because of the counterparts, violating the first premise does not result in a type error. It instead leads to a different typing rule being applied. On the other hand, violating the seventh premise yields a type error, as no other premise in the type system specifies its counterpart. Thus, the seventh premise is a dangling premise.

The key distinction between the first and seventh premises is the existence of counterparts in other typing rules. Premises with counterparts serve as control-flow filters that restrict the applicability of each typing rule to a specific set of constructs. On the other hand, premises without counterparts, such as the seventh premise, are considered assertions that must hold. Otherwise, type check fails. Based on this distinction, we identify dangling premises as those that do not have counterparts in other typing rules. We implement analysis and transformation over our mechanization to automate such identification.

We further extend SpecTec to execute the P4 type system, by implementing an interpreter that runs the mechanized type system. Because SpecTec does not generate a P4 parser from the mechanized syntax definition (yet), we instead adapt the Petr4 [12] parser, a P4 parser in OCaml. Given a P4 program as input, we parse the program and feed it to the mechanized typing rules. Running the interpreter on the typing rules yields a typing result, either well-typed or ill-typed. Interpretation also computes the dangling coverage, the set of dangling premises that the program covers. By *covered*, we mean that the dangling premise was *violated*, i.e., an ill-typed condition was satisfied. This draws a correspondence between the ill-typed conditions and the P4 test programs. Details of identifying dangling premises and executing the type system are discussed in Section 4.



(a) Overview of the mechanized P4 type system.



(b) Overview of the negative test fuzzing process.

Fig. 2. Overview of our approach.

2.2 Fuzzing Process

Once we have related P4 programs to the ill-typed conditions via mechanization, we apply fuzzing to generate negative tests (Fig. 2b). We implement a fuzzer as a backend of the SpecTec framework, which mutates well-typed P4 programs to generate ill-typed programs that increase the dangling coverage. Consider the well-typed P4 program **`shift-well.p4`**:

```
header Hdr { bit<8> src; }
bit<32> f(in bit<32> x, in Hdr hdr) { return x << hdr.src; }
```

This can be mutated to **`shift-ill.p4`** by changing the type of the `src` field from `bit<8>` to `int<8>`.

The fuzzer takes a set of well-typed P4 programs as seeds, and its dangling coverage as input. Fuzz Driver iterates over the uncovered dangling premises and selects a seed program correlated to each dangling premise. Then Program Mutator mutates the selected program, aiming to generate an ill-typed program that covers the dangling premise. The mutated program is executed by the Executable P4 Type System (Fig. 2a). It checks whether the mutated program is ill-typed and covers a new dangling premise. If so, the mutated program is added to the set of ill-typed programs. The fuzzer continues this process until a user-provided loop bound is reached. We output the final set of generated ill-typed programs as the result. Details of the fuzzer are discussed in Section 5.

```
syntax id = text
syntax dir = IN | OUT | INOUT
syntax binop = PLUS | MINUS | SHL | SHR
syntax type =
 | NameT id | IntT | FIntT expr | FBitT expr

syntax expr =
 | NameE id | IntE int
 | FIntE nat int | FBitE nat int
 | BinE binop expr expr | ExprAccE expr id
syntax stmt = RetS expr?
syntax decl =
 | HeaderD id (id, type)*
 | FuncD id type (id dir type)* stmt*
syntax program = decl*
```

(a) Simplified P4 abstract syntax in SpecTec.

```
syntax typeIR =
  | IntT | FIntT nat | FBitT nat
  | HeaderT id (id, typeIR)*

syntax annotIR = `( typeIR )
syntax exprIR =
  | NameE id annotIR | IntE int annotIR
  | FIntE nat int annotIR
  | FBitE nat int annotIR
  | BinE binop exprIR exprIR annotIR
  | ExprAccE exprIR id annotIR
```

(b) P4 intermediate representation in SpecTec.

Fig. 3. P4 abstract syntax and intermediate representation in SpecTec.

3 Mechanized P4 Type System

Mechanizing the P4 type system involves two steps: (1) defining a formal P4 type system and (2) mechanizing it using the SpecTec framework. First, to define a formal P4 type system, we build upon previous formalization efforts including Petr4 [12], P4Cub [24], and HOL4P4 [4]. Reusing these as-is is not possible, as they cover only a subset of the full P4 language and are not up-to-date with the latest specification. We extend them to cover the full P4 syntax, excluding P4 annotations and recently added features such as for loop, up to the current version, v1.2.5.

Next, we mechanize our formalization using the SpecTec framework. SpecTec was developed to automate the WebAssembly (Wasm) standardization process. Its meta-language is an ASCII representation of typical pencil-and-paper formalizations. From a mechanized Wasm specification defined in the SpecTec meta-language, SpecTec generates a formal and prose specification, and a Wasm interpreter as backends.

The SpecTec meta-language is generic enough to define the P4 type system in big-step inference rules. Yet, as SpecTec was designed for Wasm, its backends are tailored to Wasm semantics: the interpreter backend assumes small-step semantics, incompatible with our formalization. Thus, we adapt SpecTec to support execution of the big-step P4 type system.

Now, we describe a simplified version of our mechanized type system. Note that there are two languages at play: the SpecTec meta-language and P4. To distinguish them, we prefix *meta-* when referring to the SpecTec meta-language.

P4 Abstract Syntax. Fig. 3a shows the P4 abstract syntax in SpecTec. `nat`, `int`, and `text` are primitive meta-types for natural numbers, integers, and strings, respectively. Keyword `syntax` defines a meta-type. Here, `type` is a meta-type for P4 types with four variants: named (**`NameT`**), arbitrary-precision signed integer (**`IntT`**), fixed-width signed integer (**`FIntT`**), and fixed-width unsigned integer (**`FBitT`**) types. P4 constructs fall into three main syntactic categories: expressions (`expr`), statements (`stmt`), and declarations (`decl`). In the definitions of `stmt` and `decl`, `*` and `?` denote a possibly non-empty sequence and an optional element, respectively.

The program **`shift-well.p4`**, after parsing, becomes a meta-value of meta-type `program`. It is a sequence of two meta-values **`HeaderD`** and **`FuncD`**. Inside **`FuncD`**, a return statement **`RetS`** contains a binary expression **`BinE`** with `(`**`ExprAccE`** `(`**`NameE`** `"hdr") "src")` as its right operand.

```
relation Expr_ok:
  context |- expr : exprIR hint(input %0 %1)

dec $compatible_shift(typeIR, typeIR) : bool
def $compatible_shift(FBitT nat_a, IntT) = true
def $compatible_shift(FBitT nat_a, FIntT nat_b) = true
def $compatible_shift(FBitT nat_a, FBitT nat_b) = true
def $compatible_shift(typeIR_a, typeIR_b) = false
 -- otherwise

dec $is_ctk_non_neg(context, exprIR) : bool
;; implementation omitted : true if exprIR is compile-time known and non-negative
```

Fig. 4. Expression typing relation and auxiliary meta-functions.

P4 Intermediate Representation (IR). For typing, we define a P4 IR that extends the abstract syntax with additional information. Its syntax is shown in Fig. 3b. Notice in `typeIR` that named type (**`NameT`**) from `type` is resolved to its corresponding type, which includes a header type **`HeaderT`**. `exprIR` carries `annotIR`, which records the IR type of the expression.

Typing Relation and Meta-Functions. The signature of the expression typing relation is shown at the top of Fig. 4. Though omitted here, `context` represents a typing context, which includes a map from variable names to types. A **`hint`** next to the signature tells which part of the relation acts as the input. `(input %0 %1)` means that the first two arguments, `context` and `expr`, are the inputs to the relation. While relations in general are simply sets of ordered tuples, this hint restricts its interpretation to have designated inputs and outputs. Such restriction later enables SpecTec to execute the typing relations. Auxiliary meta-functions can also be defined in SpecTec. In Fig. 4, we show meta-functions `$compatible_shift` and `$is_ctk_non_neg`. Keyword **`dec`** declares the meta-function signature, and **`def`** provides its implementation.

Typing Rule. **`relation`** `Expr_ok` is a collection of typing rules for P4 expressions. Considering `Expr_ok` as an algorithm for typing expressions, each typing rule represents a distinct execution path in the algorithm. Thus, typing rule in Fig. 1 expresses an execution path for typing shift expressions. We walk through this using program **`shift-well.p4`** as an example.

```
rule Expr_ok/bine-shift:
  C |- BinE binop expr_l expr_r : BinE binop exprIR_l exprIR_r `( typeIR_l )
```

The typing context `C` and a binary operator expression are the inputs to the rule. At this point, the context maps `x` to (**`FBitT`** `32`) and `hdr` to (**`HeaderT`** `"Hdr" [("src", FBitT 8)]`).

```
-- if binop <- [ SHL, SHR ] ;; 1
-- Expr_ok: C |- expr_l : exprIR_l ;; 2
-- Expr_ok: C |- expr_r : exprIR_r ;; 3
```

The first **`if`** premise is a membership check for shift operators. The next two premises type check each operand under `C` via the relation `Expr_ok`. These are called **`rule`** premises.

```
-- if typeIR_l = $typeof(exprIR_l) ;; 4
-- if typeIR_r = $typeof(exprIR_r) ;; 5
-- if $compatible_shift(typeIR_l, typeIR_r) ;; 6
-- if (($is_intt(typeIR_r) \/ $is_fintt(typeIR_r)) => $is_ctk_non_neg(C, exprIR_r)) ;; 7
```

`$typeof` extracts the IR types of the typed operands and `$compatible_shift` checks that the operand types are compatible with shift operations. The last premise checks that if the right operand is a

```
relation Expr_ok: C |- BinE binop expr_l expr_r
 1. If binop <- [ SHL, SHR ],
   a. Expr_ok: C |- expr_l : exprIR_l
   b. Expr_ok: C |- expr_r : exprIR_r
   c. Let typeIR_l be $typeof(exprIR_l)
   d. Let typeIR_r be $typeof(exprIR_r)
   e. If $compatible_shift(typeIR_l, typeIR_r),
     i. If (($is_intt(typeIR_r) \/ $is_fintt(typeIR_r)) => $is_ctk_non_neg(C, exprIR_r)),
       1. Result in BinE binop exprIR_l exprIR_r `( typeIR_l )
```

(a) Structured control flow for shift expression typing.

```
relation Expr_ok: C |- BinE binop expr_l expr_r
 1. If binop <- [ PLUS, MINUS ], ...
```

(b) Structured control flow for plus and minus expression typing.

Fig. 5. Structured control flows before merging.

signed integer, it must be compile-time known. For **`shift-well.p4`**, this implication is vacuously true. Finally, expression typing succeeds, resulting in a typed IR expression.

Note that we applied the premises sequentially in top-to-bottom order. This is another restriction we impose on the interpretation of inference rules, where in general, premises need not be ordered. Sequential ordering eases the analysis of the typing rules, as will be discussed in the next section.

4 Dangling Premises and Coverage

Premises in the typing rules define the well-typed conditions of a P4 program. In contrast, their violations correspond to ill-typed conditions. With this observation, we identify *dangling premises*, which are conditional premises that cause type errors when violated.

4.1 Identifying Dangling Premises

Not all premises in the mechanized type system are dangling. Recall the shift expression typing rule in Fig. 1. Premises 2, 3, 4, and 5 serve to bind meta-variables, and thus are not conditions that can be violated. The first premise, although is a condition check, only narrows the scope of the rule to shift operators. Violating it does not cause the typing to fail, as there are other rules handling other binary operators. Thus, intuitively, premises 6 and 7 are the only dangling premises.

We perform two steps to identify dangling premises in the mechanized type system: a) *binding analysis* to filter out binding premises, and b) *structuring* to find counterpart premises across rules.

Binding analysis is a dataflow analysis that classifies premises into *binding* and *conditional*. `rule` premises are binding premises that introduce new meta-variables as outputs. `if` premises can have multiple interpretations. `if x = y` may mean: a comparison; binding `y` to `x`; or binding `x` to `y`.

Thus, the goal of binding analysis is to disambiguate the `=` operator in `if` premises. Starting from the inputs of the relation, the analysis incrementally tracks the set of bound variables, proceeding through each premise in order. An `if x = y` premise is disambiguated based on the bound variables: if both sides are bound, it is conditional; if one side is bound and the other is not, it is binding. The analysis rejects other cases, requiring the specification to be rewritten so that the dataflow is clear. As a result, premises 1, 6, and 7 are classified as conditional, while the other four are binding.

```
relation Expr_ok: C |- BinE binop expr_l expr_r
 1. If binop <- [ SHL, SHR ], ...
   e. If $compatible_shift(typeIR_l, typeIR_r),
     i. If (($is_intt(typeIR_r) \/ $is_fintt(typeIR_r)) => $is_ctk_non_neg(C, exprIR_r)), ...
 2. Else, ...
```

Fig. 6. Merged control flow for shift and plus/minus rules.

We then apply structuring to determine which conditional premises are dangling. Structuring is a syntactic transformation that merges the rules of a relation into a single control flow. After merging, the premises can be jointly analyzed to find counterparts, thus identifying dangling premises.

We first transform each rule into a structured control flow, converting binding premises to `Let` instructions and conditional premises to `If` instructions. The shift expression typing rule is transformed into Fig. 5a. Similarly, the other rule for plus and minus expressions becomes Fig. 5b.

Before merging the two control flows, we should ensure that their inputs are syntactically equivalent. We apply *anti-unification* [8], a preprocessing step to normalize the inputs of the rules. Anti-unification abstracts over the syntactic differences in the inputs while preserving the shared structure. For the shift and plus/minus rules, anti-unification is unnecessary, as their inputs are already the same. However, suppose we instead want to merge two rules with differing inputs: (`BinE SHL expr_l expr_r`) and (`BinE SHR expr_l expr_r`). Anti-unification would produce (`BinE binop expr_l expr_r`) as the common input. We would then introduce auxiliary instructions, `If binop = SHL` and `If binop = SHR`, to each corresponding rule.

After anti-unification, the inputs of each structured rule are syntactically equivalent. So it is safe to merge them into a single control flow via concatenating the instructions:

```
relation Expr_ok: C |- BinE binop expr_l expr_r
 1. If binop <- [ SHL, SHR ], ...
 2. If binop <- [ PLUS, MINUS ], ...
```

After merging, distinct execution paths of each rule are combined into a single control flow representing the entire relation. We apply syntactic heuristics to group counterpart conditionals in the merged control flow. These heuristics detect patterns among consecutive `If` instructions and merge them when possible. In the example, `If` conditions of steps 1 and 2 are syntactically similar, both checking membership of `binop` in a list of operators. After identifying candidates for merging, we check that the conditions are indeed counterparts. The two operator lists, `[SHL,SHR]` and `[PLUS,MINUS]`, are disjoint, together specifying all possible cases of meta-type `binop` defined in Fig. 3a. Thus, we merge the two premises into a single `If-Else` instruction. We repeat this process until no further merging is possible, resulting in Fig. 6.

We now define a dangling premise as a conditional premise that appears as an `If` without an `Else` branch after structuring. The first premise in Fig. 1 is not dangling, as it is an `If-Else` instruction in step 1 of Fig. 6. The sixth and seventh premises, each lacking a counterpart, remains as isolated `If` instructions in steps 1.e and 1.e.i. Therefore, they are dangling premises.

We apply the same procedure to all relations and meta-functions in the mechanized P4 type system. From 2,530 premises in the mechanization, we obtain 1,225 dangling premises. We exclude those inside relations and meta-functions performing compile-time evaluation of P4 expressions. Compile-time evaluation occurs after expressions are type checked by `Expr_ok`; thus, it is irrelevant to the ill-typedness of the expressions. This leaves 939 dangling premises as our primary focus.

4.2 Caveats

Although binding analysis and structuring filter out irrelevant premises, several caveats remain.

Not Falsifiable. Redundant conditions may introduce dangling premises that cannot be violated. For example, suppose we have an additional premise in our example:

```
1. If binop <- [ SHL, SHR ],
  a. If binop =/= PLUS, ... ;; cannot evaluate to false
```

Step 1.a is dangling, but it can never evaluate to false due to step 1. Although this is a contrived example, such cases exist because redundant checks are present across relations and meta-functions.

False Positives. Structuring relies on syntactic heuristics, so it may fail to identify counterparts.

```
1. If binop <- [ SHL, SHR ], ...
2. If binop = PLUS \/ binop = MINUS, ...
```

In this example, even though the two conditions exhaustively enumerate `binop`, structuring fails to merge them due to their syntactic differences. As a result, each is left as a dangling premise. Adding more syntactic heuristics may help reduce false positives, but is insufficient to eliminate all of them.

4.3 Dangling Coverage

The structured specification is an imperative program with explicit control and data flow via `If` and `Let` instructions. We implement an interpreter in OCaml that evaluates the structured specification, analogous to our walkthrough of the typing rules. By running the interpreter, we can execute the type system on a given P4 program and determine whether it is well-typed or ill-typed.

Executability also allows us to measure which dangling premises have been covered by an input P4 program, i.e., the *dangling coverage*. A dangling premise is covered, or *hit*, if the condition of the premise evaluates to false. Otherwise, if the condition evaluates to true, or is not reached, it is *missed*. Notice that this inverts the usual intuition. Because our objective is to enumerate ill-typed conditions, hits occur on premise violations and misses occur on premise satisfactions.

We define two additional terms used in fuzzing: *close-miss* and *likely-hit*. A close-miss occurs when a dangling premise is missed because its condition was evaluated to true. That is, the condition was reached, but was not violated. These are promising fuzz candidates, as small changes may cause them to hit the dangling premise. A likely-hit is a dangling premise that is only hit by ill-typed P4 programs. Due to false positives, we cannot guarantee that every dangling premise represents an ill-typed condition. Likely-hits help filter such false positives.

5 Fuzzing Ill-Typed Programs

Our goal is to mutate well-typed P4 programs into ill-typed programs that increase the dangling coverage. We refine this goal with *close-miss*: for each dangling premise, mutate its close-missing well-typed program into an ill-typed program that hits the dangling premise. This raises a sub-goal: which parts of the program should we select for mutation?

5.1 Selecting Mutation Points

We present three answers for selecting mutation points: *random*, *derive*, and *hybrid* strategies. The simplest is *random* strategy, randomly selecting an AST node in the well-typed program. It can be inefficient, as the selected node may not contribute to the dangling premise's evaluation.

Derive strategy computes a *value-dependency graph (VDG)* during specification interpretation to derive AST nodes that contribute to the close-miss. A VDG is a directed graph $G = (V, E)$ where nodes represent meta-values, and edges capture data dependencies between them. Each

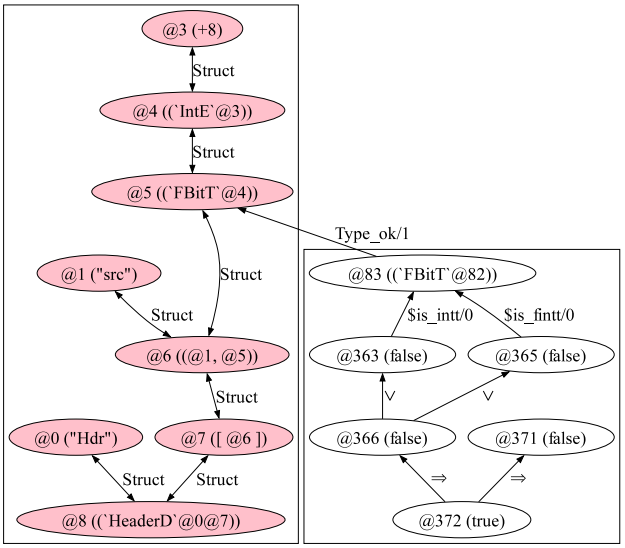


Fig. 7. (left) VDG for the header type declaration. (right) Backtracking from `@372` to find close-AST node `@5`.

node $(m, \tau, k) \in V$ is a triple with $m \in$ MetaValue, $\tau \in$ MetaType, and $k \in \{\text{source}, \text{nonsource}\}$. k indicates whether the node originates from the source program or is generated during interpretation. Each edge $(v_a, v_b, \ell) \in E$ denotes a dependency from v_a to v_b, labeled ℓ, which is one of the following:

- Struct for structural dependencies, i.e., inclusion and projection;
- Op for operator computation, e.g., addition (+) and logical implication (=>);
- Call for inputs-outputs of meta-functions and relations;
- Control for control dependencies from `If` branching structures.

The input program AST is a collection of meta-values in the VDG. On the left of Fig. 7, we show a part of the VDG for **`shift-well.p4`** representing the header type declaration. The colored nodes indicate source. For instance, node `@5` represents `bit<8>`, with a Struct edge to node `@6` for `bit<8> src;`. Each node also has a meta-type, though not shown in the figure. Node `@5` has meta-type `type`. It is possible because the interpreter keeps track of the meta-types of the meta-values.

A close-miss occurs when a dangling premise is missed because its condition evaluates to true. This means that interpreting the condition yields a true boolean meta-value. This meta-value is computed during interpretation, thus is a nonsource node in the VDG. Starting from this node, we collect all reachable source nodes in the VDG. These nodes correspond to parts of the program that likely caused the condition to evaluate to true. We refer to them as *close-AST nodes*.

Given a pair of a dangling premise and a close-missing well-typed program, we compute close-AST nodes as potential mutation points. Consider **`shift-well.p4`** and dangling premise 1.e.i in Fig. 6. **`shift-well.p4`** close-misses the dangling premise, because evaluating the condition at step 1.e.i yields true. This boolean meta-value is represented by node `@372` in the VDG in Fig. 7. Starting from `@372`, we backtrack through the VDG to find the close-AST node `@5` representing `bit<8>`.

However, backtracking may not yield any source nodes, since the VDG is not a complete representation of the value dependencies. In this case, we fall back to the random strategy. This is called *hybrid strategy*, used by default in our fuzzer.

Algorithm 1 Fuzz/Reduce Cycle

```
1: function Campaign(H_init, M_init)
2:   H_seed ← H_init, M_seed ← M_init
3:   H_fuzz ← H_init, M_fuzz ← ∅
4:   repeat
5:     Reducer(Spec, M_seed, M_fuzz)
6:     Fuzzer(Spec, H_fuzz, M_fuzz)
7:     UpdateSeed(H_seed, M_seed, H_fuzz, M_fuzz)
8:   until user-specified bound
9:   return LikelyHits(H_seed)
```

Algorithm 2 Reduce Phase

```
1: function Reducer(Spec, M_seed, M_fuzz)
2:   for each (d ↦ P_close) ∈ M_seed do
3:     p ← SelectUnreduced(P_close)
4:     p_red ← Reduce(Spec, p, d)
5:     M_fuzz(d) ← M_fuzz(d) ⋃ {p_red}
```

Algorithm 3 Fuzz Phase

```
1: function Fuzzer(Spec, H_fuzz, M_fuzz)
2:   D ← Domain(M_fuzz)
3:   repeat
4:     for each d ∈ D do
5:       P_close ← M_fuzz(d)
6:       P_sample ← Take(Shuffle(P_close), N)
7:       for each p ∈ P_sample do
8:         VDG ← ExecSeed(Spec, p)
9:         N_close ← Backtrack(VDG, d)
10:        N_sample ← Take(Sort(N_close), M)
11:        for each n ∈ N_sample do
12:          repeat
13:            p_mut ← ProgramMutator(p, n)
14:            wt, D_hit, D_miss ← ExecMut(Spec, p_mut)
15:            UpdateFuzz(H_fuzz, M_fuzz, wt, D_hit, D_miss)
16:          until K times
17:   until user-specified bound
```

5.2 Close-AST Mutation

Given a close-AST node, we sample its sub-AST and apply one of the following three mutations:

1) Meta-type Driven Mutation. Since the VDG captures the meta-types of the meta-values, we may replace the sub-AST with a random meta-value of the same meta-type. Following QuickCheck [10], we derive a random meta-value generator per meta-type definition. From the definition of `type`, SpecTec derives a generator producing random meta-values of the forms **`NameT`**, **`IntT`**, **`FIntT`**, and **`FBitT`**. Thus, node `@5` in Fig. 7 can be replaced with **`FIntT`**, yielding **`shift-ill.p4`**. The base cases of this mutation are the primitive meta-types, `bool`, `nat`, `int`, and `text`. For `text`, we sample from a pool including identifiers that were declared in the P4 program.

2) List Mutation. A list meta-value is mutated by randomly inserting duplicates, removing elements, or shuffling elements. For instance, a singleton list `@7` containing `@6` can be mutated to `[@6,@6]` or an empty list `[]`.

3) Constructor Mutation. There are similar variants within a meta-type, having the same fields but differing only in the constructor, e.g., **`FBitT`** `expr` and **`FIntT`** `expr`. For these groups of similar variants, we replace a constructor with another to yield subtly different P4 constructs.

5.3 Fuzz/Reduce Cycle

We finally combine the executable mechanized type system, dangling coverage, and close-AST mutation to implement a fuzz campaign. It is defined using the following components:

Spec	mechanized type system specification
$d \in D$	set of dangling premises
$p \in P$	set of P4 programs
$n \in \mathcal{N}$	set of AST nodes
$H : D \to \mathcal{P}(P)$	map for hit dangling premises where $H(d)$ are hitting programs
$M : D \to \mathcal{P}(P)$	map for missed dangling premises where $M(d)$ are close-missing programs

Alg. 1 presents the fuzz campaign. Campaign takes as input the initial dangling coverage H_{init} and M_{init} of the seed well-typed programs. It alternates between reducing (Alg. 2) and fuzzing

(Alg. 3) phases until a user-specified bound is reached. The reducing phase shrinks close-missing programs to: (1) narrow down the search space for close-AST nodes and (2) reduce the runtime of the executable type system during the fuzzing phase. The fuzzing phase takes as input the reduced close-miss programs, and mutates them to generate ill-typed programs.

The campaign keeps track of four maps: H_{seed}, M_{seed}, H_{fuzz}, and M_{fuzz}. H_{seed} and M_{seed} record total dangling coverage throughout the campaign. They include the unreduced programs in the initial seed. The reducing phase picks unreduced programs from M_{seed}, reduces them, and adds the reduced programs to M_{fuzz}. Thus, M_{fuzz} contains only reduced programs. In the fuzzing phase, reduced close-miss programs in M_{fuzz} are mutated to generate ill-typed programs. If a mutated program hits a new dangling premise, H_{fuzz} is updated accordingly.

Campaign calls Reducer at line 5. For each d in M_{seed}, it selects one unreduced program p in $M_{\text{seed}}(d)$ and reduces it to p_{red}. We use C-Reduce [25], as the P4 syntax resembles C. We pass an interestingness criterion to C-Reduce, such that p_{red} retains the property that it close-misses d. After reducing, in line 5 of Reducer, we extend M_{fuzz} with the reduced program p_{red}.

Campaign then calls Fuzzer at line 6, which repeatedly runs fuzz iterations. In Fuzzer, lines 4-6 iterate over the missed dangling premises d in M_{fuzz} and sample N close-missing programs per d. ExecSeed at line 8 executes the type system on each sampled close-missing program to compute its VDG. We find the close-AST nodes by backtracking and take the top M close-AST nodes, sorted by relevance to the dangling premise (lines 9-10). In lines 11-16, for each close-AST node n, we mutate it K times. ExecMut runs the type system on each mutated program, returning its well-typedness wt as well as hit and close-missing dangling premises D_{hit} and D_{miss}. Based on the result, we update the coverage maps H_{fuzz} and M_{fuzz}. Because the fuzzing campaign is designed to mutate close-missing programs that are well-typed, we update M_{fuzz} only when wt is true. Thus, in a single iteration, we apply $N \cdot M \cdot K$ mutations per missed dangling premise.

After the fuzzing phase, Campaign updates the seed maps H_{seed} and M_{seed} from H_{fuzz} and M_{fuzz} at line 7. Alternating between the two phases for a user-specified number of iterations yields a final coverage map H_{seed}. We filter H_{seed}, only taking dangling premises that are only hit by ill-typed programs – the *likely-hits*. This helps reduce the number of false positives in the final coverage map. Finally, we return the ill-typed programs for the likely-hits as the output of the fuzz campaign.

6 Evaluation

We evaluate our work with the following research questions:

RQ1. Coverage of Generated Tests
Does the fuzzer achieve greater dangling coverage than the P4C test suite? (6.2)

RQ2. Effectiveness of VDG and Reducer
Do VDG and Reducer help efficient negative test generation? (6.3)

RQ3. Bug Finding Ability
Does the fuzzer find new bugs in the P4C compiler? (6.4)

RQ4. Adequacy of Dangling Coverage
Does dangling coverage capture subtle ill-typed conditions, showing greater sensitivity to the negative input space than branch coverage on the P4C compiler source? (6.5)

RQ5. Comparison with Gauntlet
Can Gauntlet, the state-of-the-art P4 fuzzer, systematically generate negative tests? (6.6)

6.1 Implementation and Experimental Setup

Our implementation extends the OCaml codebase of SpecTec. We removed Wasm-specific backends and added binding analysis, structuring, and a specification interpreter.

Table 1. Runtime and reduction/generation counts.

	Reduce Phase		Fuzz Phase		
	Time (s)	Reduced	Time (s)	Ill-Typed	Well-Typed
Cycle 1	8,703	748	663	389	1
Cycle 2	3,610	262	1,287	14	0
Cycle 3	3,117	224	2,264	8	0
Cycle 4	3,091	213	2,300	8	0
Cycle 5	3,202	200	2,544	3	0
Total	21,723	1,647	9,058	422	1

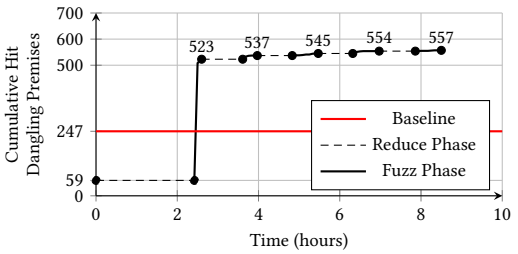


Fig. 8. Cumulative hit dangling premises over time. The total number of dangling premises is 939.

The mechanized P4 type system consists of 101 relations with 511 rules, and 379 meta-functions implemented by 1,081 `def`s. Structuring the specification yields 1,225 dangling premises. We excluded those in relations and meta-functions performing compile-time evaluation of P4 expressions, leaving 939 dangling premises as targets for the fuzzer.

Driver for the Fuzz/Reduce cycle in Alg. 1, 2, and 3 is implemented in Python. The fuzzer is implemented as a backend of SpecTec. Each fuzzing phase iterates the fuzz loop 10 times. We set $N = 3$, $M = 10$, and $K = 5$, so that each iteration generates 150 programs per dangling premise. The reducer uses the latest version of C-Reduce with the `-not-c` option to skip C-specific passes. We set a timeout of 25 seconds per reduction.

The P4C compiler test suite serves as both the baseline and seed for fuzzing. Experiments use P4C version 1.2.5.6, the latest release as of May 2025. In `testdata` directory of the P4C repository, `p4_16_samples` contains 1,278 positive tests, and `p4_16_errors` contains 310 negative tests. We excluded programs in the following categories:

- Features not yet standardized, e.g., for loops (33).
- Compile target-specific features, e.g., annotations (59).
- Features under discussion with the language design working group or P4C developers (68).

We also excluded 4 well-typed and 4 ill-typed programs due to limitations in our mechanized type system, such as timeouts on large integer literals and a bug in the Petr4 parser. In total, we excluded 112 positive and 56 negative tests, resulting in 1,166 positive and 254 negative applicable tests. A complete list of excluded programs is available in the artifact.

We label the P4C test suite after exclusion as *Baseline* and use its positive tests as the seed for the fuzzing campaigns in the following sections. We validated that our mechanized P4 type system passes all of the Baseline tests, adding confidence to our mechanization's correctness.

All experiments were conducted on an Ubuntu machine with 4.0GHz Intel(R) Core(TM) i7-6700k and 32GB of RAM (Samsung DDR4 2133MHz 8GB*4).

6.2 RQ1: Coverage of Generated Tests

We run 5 Fuzz/Reduce cycles to generate negative tests. Table 1 shows the runtime and number of programs generated or reduced per cycle. The campaign fuzzes 422 ill-typed programs and reduces 1,647 close-missing programs over 8.5 hours. Due to false positives, the fuzzer also generates 1 well-typed program that covers dangling premises.

Fig. 8 shows the cumulative number of hits against time. The horizontal line indicates Baseline. The solid and dashed lines each indicate the fuzz and reduce phases. From the initial 59 hits (6.28%), the fuzzer achieves 557 hits (59.32%), surpassing Baseline with 247 (26.30%). Among the 557 hits, 497 are likely-hits, which are likely *not* false positives. In contrast, Baseline has only 188 likely-hits. Baseline and the campaign share 174 likely-hits. Baseline has 14 unique likely-hits, while the

Table 2. Fuzz campaigns labeled C1-C6, varying along reducer usage and selection strategy. Fuzz/Reduce Cycle column shows the cumulative number of hit dangling premises per cycle. The last three columns show the final cumulative number of hits, likely-hits, and targeted hits.

Label	Reducer usage	Selection strategy	Fuzz/Reduce Cycle 1	2	3	4	5	6	7	Hits	Likely-hits	Targeted hits
C1	With	Derive	527	540	545	551	555	555	555*	555	494	65
C2		Random	520	538	545	556	557	560	560*	560	498	49
C3		Hybrid	522	538	546	551	553	554	554*	554	494	63
C4	Without	Derive	544†	–	–	–	–	–	–	544	483	51
C5		Random	528†	–	–	–	–	–	–	528	467	16
C6		Hybrid	542†	–	–	–	–	–	–	542	480	48

* Timeout during reducing phase, † Timeout during fuzzing phase

campaign has 323 unique likely-hits. Thus, the generated programs cover a wider range of ill-typed conditions than Baseline.

We manually inspected the number of genuine ill-typed conditions among the likely-hits and the misses. A dangling premise is either: a true positive, representing a genuine ill-typed condition; a false positive; or not falsifiable. We classified a dangling premise as a false positive if we could not correlate it to a concrete ill-typed condition. Reasoning about not falsifiable cases is non-trivial, as it requires reasoning that no P4 program can falsify it. We thus conservatively labeled premises as not falsifiable only when it was evident from the nearby premises. Of the 497 likely-hits, there were 29 false positives, thus hitting 468 true positives. Of the 382 misses, 205 were true positives, that is, the genuine ill-typed conditions that were not violated. These misses depend on the fuzzer seed, mutation strategies, and time budget. Our notion of dangling coverage quantifies the limitations of the current P4C negative tests, and our mutation strategy on positive P4C tests already hits a large portion of the dangling premises. Uncovering the remaining ones is left for future work.

6.3 RQ2: Effectiveness of VDG and Reducer

Both the VDG and the reducer are optimizations to reduce the fuzzer's search space. By backtracking dependencies in the VDG, the fuzzer can pinpoint the AST nodes that would likely produce a hit when mutated. The reducer acts as a preprocessor for the fuzzer, shrinking the program ASTs.

To evaluate their effectiveness, we conduct six fuzz campaigns labeled C1-C6 in Table 2, varying along reducer usage and AST node selection strategy. Campaigns C1-C3 use the reducer; C4-C6 do not. Each group uses three selection strategies: derive, random, and hybrid. All campaigns run with a 12 hour timeout. Sorted by final number of hits in Table 2:

$$C5 < C6 < C4 < C3 < C1 < C2$$

The trend is similar when sorted by likely-hits. Comparing C1-C3 with C4-C6, we observe that the reducer improves dangling coverage. It is notable that C4-C6 without the reducer reach timeouts in the first cycle, showing that the reducer speeds up fuzzing by shrinking program size.

To assess the VDG, we first compare within C1-C3 and C4-C6. Among C4-C6, C4 and C6 (derive and hybrid) cover more dangling premises than C5 (random), showing that VDG effectively narrows the search space on unreduced programs. Within C1-C3, C2 (random) achieves the highest coverage. We interpret this as the reducer's effect dominating that of the VDG. Since the reducer significantly reduces program size, the search spaces for random and derive become comparable.

For a closer examination of the VDG, we compare the number of *targeted hits*. A targeted hit occurs when the fuzzer hits the dangling premise it currently targets. The fuzz phase in Alg. 3 iterates over each target dangling premises d (line 4). The VDG identifies mutation points that likely contribute to hitting d. However, notice that the algorithm updates the coverage whenever a

Table 3. Compiler bugs discovered in fuzzing campaigns. N/A indicates bugs found during development.

ID	Issue#	Campaign	Description
CB1	5286	C1, C2	SIGSEGV when analyzing malformed struct type.
CB2	5292	C1-C3	SIGSEGV on malformed struct initializer.
CB3	5296	N/A	SIGSEGV on malformed `__v1model_version` initializer.
CB4	5267	C1-C5	"Null srcType" on malformed enum field.
CB5	5287	C2	"Null dotsField" when struct initializer specifies a non-existent field with default initializer `(...)`.
CB6	5288	N/A	"Unknown destination type" when type inference yields a malformed type.
CB7	5291	N/A	"Could not find type of <Property>" when table default property shadows a match-action name.
CB8	5293	C1-C3, C5	"Size not yet known" on initializing malformed header stack.
CB9	5294	C1	"expected no type variables" when a method's type parameter shadows a global typedef.
CB10	5295	N/A	"Cannot find declaration" when instance name is malformed.
CB11	5297	N/A	"type names should not appear in unification" when accessing a non-existent enum field.
CB12	5375	N/A	"could not find type of unknown struct" when type inference fails on record expression.
CB13	5376	C2, C3	"should not infer new types anymore" when tuple type contains don't care type.
CB14	5378	C6	"Null resolvedType" when a serializable enum's underlying type is ill-formed.

Table 4. Soundness bugs discovered in fuzzing campaigns. N/A indicates bugs found during development.

ID	Issue#	Campaign	Description
SB1	5253	C1, C3-C6	(7.2.10.2) Fail to reject duplicate constructor declarations.
SB2	5254	C1-C6	(12.7.3) Fail to reject duplicate switch labels.
SB3	5255	N/A	(Appendix F) Fail to reject a function calling an extern function.
SB4	5256	C1-C6	(7.2.8) Fail to reject invalid type nesting.
SB5	5265	C1-C6	(8.13) Fail to reject when default struct initializers specify more values than the number of fields in the struct.
SB6	5266	C2, C3, C5	Fail to reject type mismatch between parameter type and its default value.
SB7	5299	C1-C3	(8.17) Fail to reject argument mismatch in built-in method call to `isValid()`.
SB8	5300	C2, C3	(8.8) Fail to reject bitwise operation on arbitrary precision integers.
SB9	5301	N/A	(8.11.1) Fail to reject explicit cast from `int<X>` to `bit<Y>` when `X != Y`.
SB10	5302	C1-C3	(8.6-8) Fail to reject out-of-bounds bitslice (`[H:L]`) before compile-time evaluation of constant expressions.
SB11	5303	N/A	Fail to insert implicit casts before compile-time evaluation of constant expressions.
SB12	5304	C1	(13.5) Fail to reject parser state transition to a non-existent parser state.
SB13	5382	N/A	(12.7.2) Fail to reject switch statement on string literals.
SB14	5383	C1, C3, C6	(11.3.1) Fail to reject type parameter mismatch between abstract method declaration and implementation.
SB15	5385	C4	(6.8) Fail to reject a parameter of arbitrary precision integer type having a direction.

mutated program hits new dangling premises in D_{hit}, regardless of whether it contains the target d (line 15). Thus, to better assess the VDG, we count targeted hits, occurring when $d \in D_{hit}$. As Table 2 shows, C4 and C6 outperform C5 in targeted hits, and likewise for C1 and C3 over C2. This suggests that the VDG effectively guides the fuzzer toward hitting the targeted dangling premises.

6.4 RQ3: Bug Finding Ability

We ran `p4test`, the type checker frontend of P4C, on the ill-typed programs from campaigns C1-C6. While the expected behavior is to reject all ill-typed programs, there were cases where it unexpectedly terminated or accepted the programs. We categorized them as *compiler bugs* and *soundness bugs*, respectively. Compiler bugs are bugs that cause P4C to crash or output "Compiler Bug". "Compiler Bug" indicates an abrupt assertion failure, which is understood as a bug by the P4C developers. Soundness bugs are the cases where P4C fails to reject the ill-typed programs.

We discovered 14 compiler bugs and 15 soundness bugs. Table 3 summarizes the compiler bugs labeled **CB1-CB14**. The campaign column indicates the fuzzing campaigns in which each bug was found, where N/A indicates bugs found during development of the fuzzer. Three bugs cause SIGSEGV (**CB1-CB3**). Bugs arise in corner cases of the type system, such as struct initializers (**CB2**, **CB5**), shadowing (**CB7**, **CB9**), and type inference (**CB6**, **CB12**). All compiler bugs have been confirmed by the P4C developers.

Table 5. P4C source branch coverage and dangling coverage on Baseline and Section 6.2 campaign results.

Metric	Baseline			Campaign		
	Pos	Neg	Total	Pos	Neg	Total
Branch(%)	34.41	20.38	35.83	34.41	13.91	35.47
Dangling(%)	6.28	25.03	26.30	6.39	58.25	59.32

Table 4 summarizes the soundness bugs labeled **SB1**-**SB15**. The description lists the section in the official P4 specification that each bug fails to comply with. For example, **SB8** violates Section 8.8, which states that *"bitwise-operations are not defined on expressions of type int."* We reported these soundness bugs to the P4C developers. **SB2** has been patched in P4C. **SB3** led to a patch in the official P4 specification. After discussion within the P4 community, it was concluded that the specification was overly strict on function call sites, so the specification was amended to relax some of the restrictions. Others are confirmed as soundness bugs and are waiting to be fixed.

These bug findings suggest that dangling coverage effectively guides the fuzzer toward diverse and subtle ill-typed conditions that can be overlooked by compiler developers.

Tests that were correctly rejected by P4C are also useful to help prevent future soundness bugs. After linting and assigning appropriate names, we integrated 248 negative tests into the P4C test suite [16]. These tests are now run as part of the P4C continuous integration workflow.

6.5 RQ4: Adequacy of Dangling Coverage

Dangling coverage aims to capture the ill-typed conditions in the P4 type system. To assess its sensitivity to the negative input space, we compare dangling coverage on the mechanized P4 type system with branch coverage on the P4C compiler source. For each Baseline and the campaign results from Section 6.2, we measure both dangling and branch coverage. The campaign results contain 1,166 well-typed programs from the seed plus 1 well-typed program and 422 ill-typed programs generated by the fuzzer. We used gcov [15] to measure branch coverage of files related to type checking in `frontends/common`, `frontends/p4`, and `midend` directories, totaling 57,874 branches.

Table 5 summarizes the results. For branch coverage, the total coverage of Baseline and the campaign differs only by 0.36%p. Comparing branch coverage of positive and negative tests within each group, positive tests exhibit higher coverage than negative tests (34.41% vs. 20.38% and 34.41% vs. 13.91%). Moreover, positive tests dominate the total branch coverage. The negligible difference in total branch coverage is consistent with related work on type checker testing [9], which reports *"traditional code coverage metrics are too shallow to capture the efficacy of our approach."*

In contrast, with dangling coverage, the campaign achieves 33.02%p higher total coverage than Baseline. Within each group, negative tests show higher coverage than positive tests (25.03% vs. 6.28% and 58.25% vs. 6.39%), and also dominate the total coverage. This contrast suggests that while branch coverage cannot adequately distinguish ill-typed programs from well-typed ones, dangling coverage can. Together with the bug finding ability of the generated tests, dangling coverage is an adequate metric for systematically exploring the negative input space of the P4 type system.

6.6 RQ5: Comparison with Gauntlet

We compare our approach with Gauntlet [27], the state-of-the-art P4 fuzzer, integrated into the P4C compiler infrastructure. Specifically, we use its program generator, P4Smith [1], to examine whether it can systematically generate diverse negative tests.

Two minor adjustments were made to P4Smith. According to its documentation, P4Smith aims to *"generate programs that are valid according to the latest version of the P4 specification."* However, it generates division and modulo operations on fixed-width integers, which is still under discussion

within the P4 community [33]. It also generates bitwise operations on arbitrary precision integers, which correspond to one of the soundness bugs (**SB8**) discussed in RQ3. Thus, we disabled their generation in P4Smith. Additionally, because P4Smith generates large 128-bit integer literals that cause timeouts in our implementation, we restricted the size of generated integers.

Running P4Smith for 12 hours produced 36,140 programs, all of which were well-typed. The programs achieve 2.98% (28/939) dangling coverage, which is lower than that of both Baseline and our fuzzing campaign in RQ1. The results indicate that P4Smith, geared towards generating valid P4 programs, is not effective in systematic generation of ill-typed programs.

6.7 Threats to Validity

Threats to internal validity arise from type system mechanization. Our approach assumes that the mechanization faithfully reflects the P4 specification. It passes most P4C tests, but any error in the mechanization can lead to misidentified dangling premises. Also, dangling coverage is sensitive to the type system's structure. Different formalizations may yield different dangling premises.

Threats to external validity stem from using the P4C positive tests as the fuzzer seed. Any bias in their construction, such as over-representation of certain P4 features, may affect program diversity.

Threats to construct validity arise from using P4C as a proxy for evaluating the adequacy of dangling coverage. The observed bug findings and coverage comparisons do not guarantee that dangling coverage precisely captures all ill-typed conditions. However, we believe P4C is a reasonable proxy, given its status as the de facto reference P4 compiler.

7 Related Work

Negative Type Checker Test Generation. There is little work on generating negative type checker tests. Dewey et al. [11] apply constraint logic programming (CLP) to test the Rust type checker. They encode fragments of the Rust type system in Prolog and use its solver to generate conforming programs. To generate ill-typed programs, they negate premises in the typing rules, forcing the solver to generate ill-typed programs satisfying the negated premises. Our approach is similar in targeting premise violations. However, as acknowledged by the authors, CLP-based generation has fundamental performance issues, which would not scale to the full P4 type system.

Chaliasos et al. [9] target soundness and precision bugs in JVM-based languages. They focus on bugs in object-oriented features, such as inheritance and parametric polymorphism among classes and interfaces. They generate well-typed programs and encode each program's type inheritance and generic instantiation relationships as a graph. By replacing a type in this graph with an incompatible one, they produce an ill-typed program that deliberately breaks these relationships. Similarly, we mutate well-typed programs to produce ill-typed ones. However, P4 is not object-oriented, so their approach is not directly applicable. Moreover, the P4 type system specifies subtle domain-specific constraints, such as compile-time evaluation and bit-level arithmetic.

Sotiropoulos et al. [29] synthesize test programs from real-world APIs to test Scala, Kotlin, and Groovy type checkers. They exploit real-world APIs to generate both well-typed and ill-typed programs with sophisticated type features, especially parametric polymorphism and overloading. However, P4 is a DSL without a rich standard library, limiting the applicability of this approach.

Overall, unlike these works, we aim to generate a comprehensive negative test suite that triggers diverse and subtle type errors over the entire P4 type system. We introduce dangling coverage to systematically characterize the negative input space and guide its exploration.

Apart from testing the type system with negative tests, some works generate tests targeting soundness bugs in static analyzers. Ami et al. [5] present the Mutation-Based Soundness Evaluation (μSE) framework that applies mutation testing to detect soundness bugs in Android static analyzers.

Our work is similar in using mutation to produce negative tests. Taneja et al. [30] target soundness bugs in LLVM's static analyses by computing maximally precise static analysis results with SMT solving. A soundness bug is detected when an analyzer produces a more precise result than the maximally precise one. This is conceptually similar to our approach utilizing dangling coverage, which systematically characterizes the negative input space.

P4 Program Generation. P4Fuzz [3] applies blackbox fuzzing to P4C, producing random programs from the P4 grammar. To generate well-typed programs that reach P4C backends, it applies manual filters in order to remove ill-typed programs. Gauntlet [27] combines program generation, translation validation, and model-based testing to detect crash and miscompilation bugs in P4C. Inspired by CSmith [31], it generates well-typed programs. In contrast, our fuzzer generates ill-typed P4 programs that represent diverse type errors, as highlighted in Section 6.6.

Language Mechanization Framework. Researchers have presented various language mechanization frameworks. ESMeta [2] parses the ECMAScript standard into a mechanized representation, from which it automatically generates a parser and interpreter [23], test suite [22], and static analyzer [21]. SpecTec [32] mechanizes the Wasm specification. The mechanized specification serves as a single source of truth from which both formal and prose specification, and a Wasm interpreter are derived. We adopt SpecTec, reusing its meta-language and adapting its backends for P4.

Fetscher et al. [14] apply constraint solving over mechanized typing rules in PLT-Redex to derive well-typed terms. JEST [22], a coverage-guided fuzzer based on ESMeta, generates JavaScript programs to increase node and branch coverage of the mechanized semantics. Similarly, we define dangling coverage over the mechanized P4 type system to guide generation of ill-typed programs.

8 Conclusion

We presented negative test generation for P4 type checkers based on a mechanized P4 type system. The key challenge in generating comprehensive negative tests is that neither the P4 specification nor the P4C compiler systematically characterizes the ill-typed conditions. To address the challenge, we mechanized the formal P4 type system using the SpecTec framework. Based on the mechanization, we collected dangling premises, conditional premises whose violations likely correspond to ill-typed conditions. We introduced dangling coverage, a novel metric quantifying how thoroughly a test suite exercises the ill-typed conditions. Building on this, we developed a fuzzer that mutates well-typed programs into ill-typed programs to increase dangling coverage. Our approach yields a negative test suite achieving 33.02%p higher dangling coverage than the P4C test suite and also uncovers 29 bugs in the P4C compiler. The generated negative tests are now part of the P4C test suite. The results demonstrate that language mechanization not only clarifies language semantics but also serves as a powerful enabler for effective test generation.

We believe that our approach can be applied to generating negative tests for other statically typed languages, as it does not heavily involve P4-specific strategies, aside from the use of a P4 program reducer. To instantiate our approach for another language, an executable mechanization of its type system is required. Mechanization enables the systematic collection of dangling premises, and executability allows for the measurement of dangling coverage. We plan to further investigate the applicability and generalizability of dangling coverage by mechanizing the type systems of other languages and applying our approach to them.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) (2021R1A5A1021944 and 2022R1A2C2003660) and the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (2024-00337703).

Data Availability

The artifact is available on Zenodo at [17]. It includes the mechanized P4 type system, the fuzzer for negative type checker tests, and the scripts for reproducing the experiments in this paper. The latest version is maintained as an open-source project on GitHub at [20].

References

[1] 2020. *P4Smith: An extensible random P4 program generator in the spirit of CSmith.* https://github.com/p4lang/p4c/tree/main/backends/p4tools/modules/smith

[2] 2022. *ESMeta: An ECMAScript specification metalanguage used for automatically generating language-based tools.* https://github.com/es-meta/esmeta

[3] Andrei-Alexandru Agape, Madalin Claudiu Danceanu, Rene Rydhof Hansen, and Stefan Schmid. 2021. P4Fuzz: Compiler Fuzzer forDependable Programmable Dataplanes. In *Proceedings of the 22nd International Conference on Distributed Computing and Networking* (Nara, Japan) *(ICDCN '21)*. Association for Computing Machinery, New York, NY, USA, 16–25. doi:10.1145/3427796.3427798

[4] Anoud Alshnakat, Didrik Lundberg, Roberto Guanciale, and Mads Dam. 2024. HOL4P4: Mechanized Small-Step Semantics for P4. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 102 (April 2024), 27 pages. doi:10.1145/3649819

[5] Amit Seal Ami, Kaushal Kafle, Kevin Moran, Adwait Nadkarni, and Denys Poshyvanyk. 2021. Systematic Mutation-Based Evaluation of the Soundness of Security-Focused Android Static Analysis Techniques. *ACM Trans. Priv. Secur.* 24, 3, Article 15 (Feb. 2021), 37 pages. doi:10.1145/3439802

[6] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: programming protocol-independent packet processors. *SIGCOMM Comput. Commun. Rev.* 44, 3 (July 2014), 87–95. doi:10.1145/2656877.2656890

[7] Mihai Budiu and Chris Dodd. 2017. The P416 Programming Language. *SIGOPS Oper. Syst. Rev.* 51, 1 (Sept. 2017), 5–14. doi:10.1145/3139645.3139648

[8] David M. Cerna and Temur Kutsia. 2023. Anti-unification and Generalization: A Survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*, Edith Elkind (Ed.). International Joint Conferences on Artificial Intelligence Organization, 6563–6573. doi:10.24963/ijcai.2023/736 Survey Track.

[9] Stefanos Chaliasos, Thodoris Sotiropoulos, Diomidis Spinellis, Arthur Gervais, Benjamin Livshits, and Dimitris Mitropoulos. 2022. Finding typing compiler bugs. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (San Diego, CA, USA) *(PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 183–198. doi:10.1145/3519939.3523427

[10] Koen Claessen and John Hughes. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*. Association for Computing Machinery, New York, NY, USA, 268–279. doi:10.1145/351240.351266

[11] Kyle Dewey, Jared Roesch, and Ben Hardekopf. 2015. Fuzzing the rust typechecker using CLP. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering* (Lincoln, Nebraska) *(ASE '15)*. IEEE Press, 482–493. doi:10.1109/ASE.2015.65

[12] Ryan Doenges, Mina Tahmasbi Arashloo, Santiago Bautista, Alexander Chang, Newton Ni, Samwise Parkinson, Rudy Peterson, Alaia Solko-Breslin, Amanda Xu, and Nate Foster. 2021. Petr4: formal foundations for p4 data planes. *Proc. ACM Program. Lang.* 5, POPL, Article 41 (Jan. 2021), 32 pages. doi:10.1145/3434322

[13] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. 2009. *Semantics Engineering with PLT Redex* (1st ed.). The MIT Press.

[14] Burke Fetscher, Koen Claessen, Michał Pałka, John Hughes, and Robert Bruce Findler. 2015. Making Random Judgments: Automatically Generating Well-Typed Terms from the Definition of a Type-System. In *Programming Languages and Systems*, Jan Vitek (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 383–405.

[15] GNU Project. 2025. *gcov: a Test Coverage Program.* https://gcc.gnu.org/onlinedocs/gcc/Gcov.html

[16] Seokhun Jeong. 2025. *Add 248 negative tests to p4_16_errors.* https://github.com/p4lang/p4c/pull/5369

[17] Jaehyun Lee, Seokhun Jeong, and Sukyoung Ryu. 2026. *Failing with Purpose: Dangling Coverage-Guided Negative Test Generation from a Mechanized P4 Type System.* doi:10.5281/zenodo.19508213

[18] P4 Language Consortium. 2024. *P4 Language Specification, Version 1.2.5.* https://p4.org/specs/

[19] P4 Language Consortium. 2025. *P4_16 reference compiler.* https://github.com/p4lang/p4c

[20] P4-SpecTec Team. 2026. *P4-SpecTec.* https://github.com/kaist-plrg/p4-spectec

[21] Jihyeok Park, Seungmin An, and Sukyoung Ryu. 2022. Automatically deriving JavaScript static analyzers from specifications using Meta-level static analysis. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 1022–1034. doi:10.1145/3540250.3549097

[22] Jihyeok Park, Seungmin An, Dongjun Youn, Gyeongwon Kim, and Sukyoung Ryu. 2021. JEST: N+1-version Differential Testing of Both JavaScript Engines and Specification. In *Proceedings of the 43rd International Conference on Software Engineering* (Madrid, Spain) *(ICSE '21)*. IEEE Press, 13–24. doi:10.1109/ICSE43902.2021.00015

[23] Jihyeok Park, Jihee Park, Seungmin An, and Sukyoung Ryu. 2021. JISET: JavaScript IR-based semantics extraction toolchain. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering* (Virtual Event, Australia) *(ASE '20)*. Association for Computing Machinery, New York, NY, USA, 647–658. doi:10.1145/3324884.3416632

[24] Rudy Peterson, Eric Hayden Campbell, John Chen, Natalie Isak, Calvin Shyu, Ryan Doenges, Parisa Ataei, and Nate Foster. 2023. P4Cub: A Little Language for Big Routers. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Boston, MA, USA) *(CPP 2023)*. Association for Computing Machinery, New York, NY, USA, 303–319. doi:10.1145/3573105.3575670

[25] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation* (Beijing, China) *(PLDI '12)*. Association for Computing Machinery, New York, NY, USA, 335–346. doi:10.1145/2254064.2254104

[26] Grigore Roșu and Traian Florin Șerbănuță. 2010. An overview of the K semantic framework. *The Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434. doi:10.1016/j.jlap.2010.03.012 Membrane computing and programming.

[27] Fabian Ruffy, Tao Wang, and Anirudh Sivaraman. 2020. Gauntlet: finding bugs in compilers for programmable packet processing. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, USA, Article 39, 17 pages.

[28] Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Thomas Ridge, Susmit Sarkar, and Rok Strniša. 2007. Ott: effective tool support for the working semanticist. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming* (Freiburg, Germany) *(ICFP '07)*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/1291151.1291155

[29] Thodoris Sotiropoulos, Stefanos Chaliasos, and Zhendong Su. 2024. API-Driven Program Synthesis for Testing Static Typing Implementations. *Proc. ACM Program. Lang.* 8, POPL, Article 62 (Jan. 2024), 32 pages. doi:10.1145/3632904

[30] Jubi Taneja, Zhengyang Liu, and John Regehr. 2020. Testing static analyses for precision and soundness. In *Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization* (San Diego, CA, USA) *(CGO '20)*. Association for Computing Machinery, New York, NY, USA, 81–93. doi:10.1145/3368826.3377927

[31] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) *(PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 283–294. doi:10.1145/1993498.1993532

[32] Dongjun Youn, Wonho Shin, Jaehyun Lee, Sukyoung Ryu, Joachim Breitner, Philippa Gardner, Sam Lindley, Matija Pretnar, Xiaojia Rao, Conrad Watt, and Andreas Rossberg. 2024. Bringing the WebAssembly Standard up to Speed with SpecTec. *Proc. ACM Program. Lang.* 8, PLDI, Article 210 (June 2024), 26 pages. doi:10.1145/3656440

[33] Vladimír Štill. 2024. *Allow division and modulo on fixed-width integers.* https://github.com/p4lang/p4-spec/issues/1327

[34] Vladimír Štill. 2025. *Shift by a string literal is not rejected by type checker.* https://github.com/p4lang/p4c/issues/5094

Received 2025-09-12; accepted 2025-12-22